Multi − level Linked List

−Destory List

−Time Complexity Calculation

using Recurrence Tree

<u>Original</u>

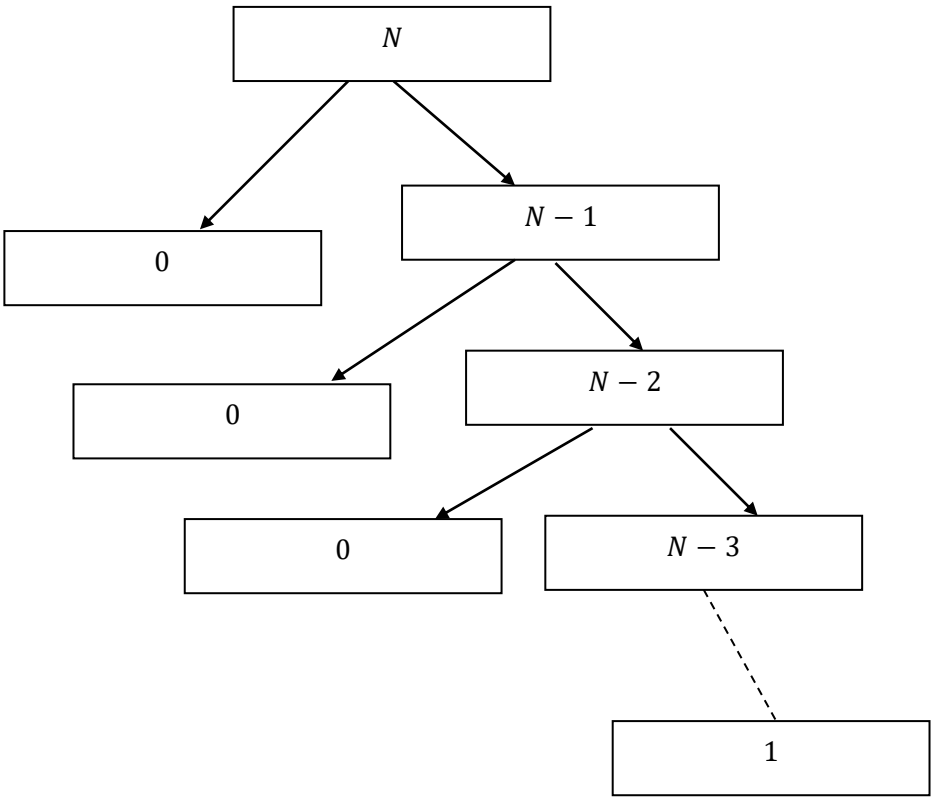
$$T(N) = \begin{cases} 1 & ,N = 0 \\ T(k) + T(N-1-k) + c & ,N > 0 \end{cases}$$

$$0 \leq k \leq N-1 \text{ and } k \neq N.$$

<u>Worst Structure</u> [***<u>For Unbalanced Tree</u>***]

$$T(N) = \begin{cases} 1 & ,N = 0 \\ T(N-1) + C & ,N > 0, k = 0 \end{cases}$$

***if we visualize the Tree*:**



if we donot consider the `0` base cases,

as, then Number of Problem doesnot increase , also it remains constant size : 1 or C.

Lets elaborate Problem Size and Work Per Problem :

Here we see : Problem Size Starts with → n

Work Per Problem or Cost Per Problem :

```
void destroyNodeRecursively(Node *node)
{
    if (node == nullptr)
        return;

    // 1. Destroy Child List first
    destroyNodeRecursively(node->child);

    // 2. Destroy Next List
    destroyNodeRecursively(node->next);

    // 3. Free current
    cout << "Freeing node: " << node->data << endl;
    free(node);
}
```

When the computer is holding a single node in its hand, it has to spend actual CPU cycles to do the following:

- ***Evaluate the if statement.***
- ***Push characters to the console screen (cout).***
- ***Talk to the operating system to release memory (free).***

And this work is being executed during runtime of recursive calls.

That physical CPU effort is the Work per Problem or Cost Per Problem.

And Cost Per Problem is : C or 1 i.e. Constant.

Similarly , for :

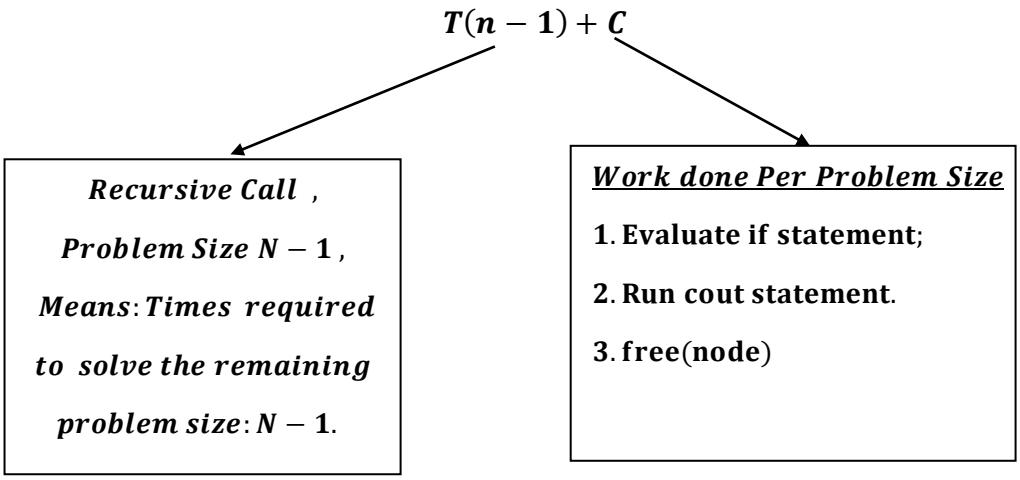
$\boldsymbol{Problem\ Size : (n-1), Cost/Work\ per\ Problem : C}$

$\boldsymbol{Problem\ Size : (n-2), Cost/Work\ per\ Problem : C}$

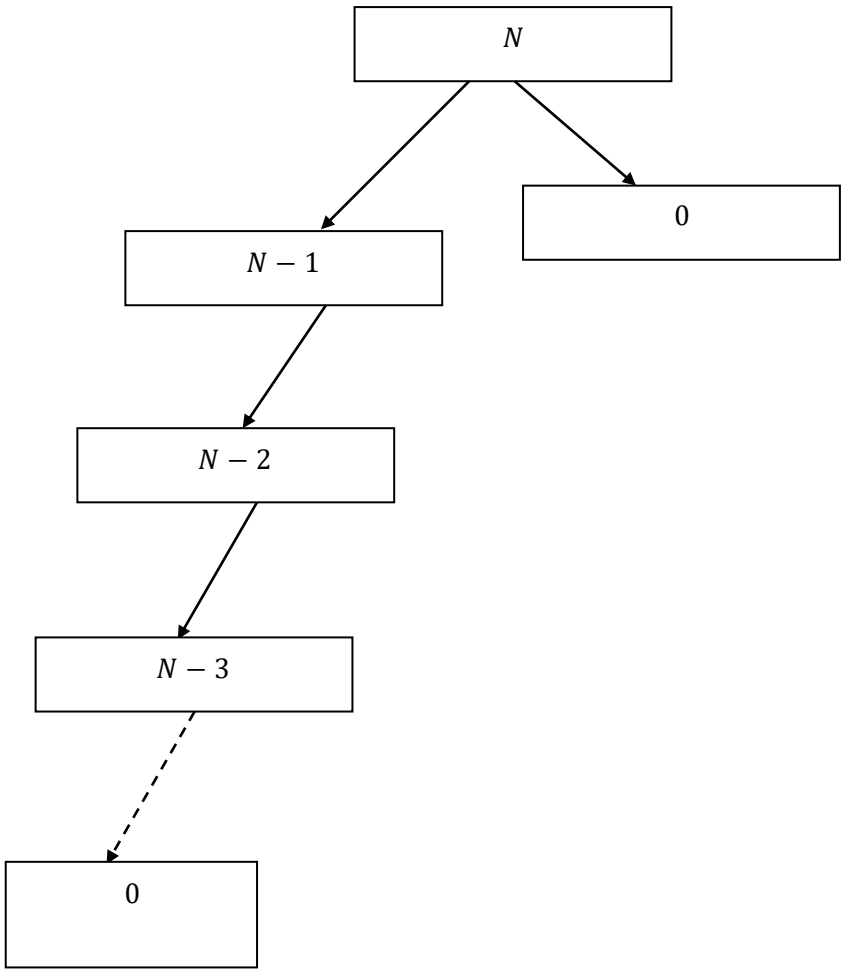
.

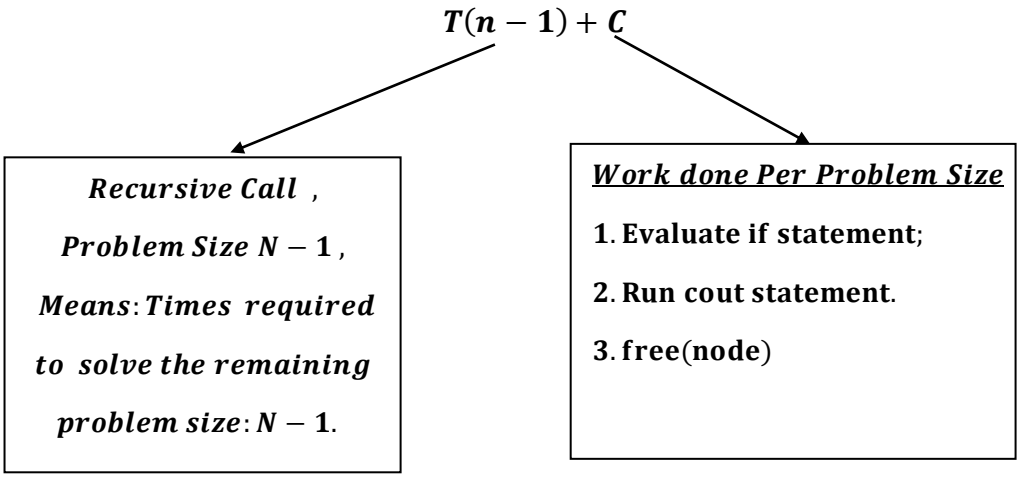
.

$\boldsymbol{Problem\ Size : 1 , Cost/Work\ per\ Problem : C}$



This is also same for :


N
0
$N-1$
$N-2$
$N-3$
0


$T(n-1)+C$
Recursive Call ,
Problem Size $N-1$,
Means: Times required to solve the remaining problem size: $N-1$.
Work done Per Problem Size
1. Evaluate if statement;
2. Run cout statement.
3. free(node)

$\boldsymbol{Level}$ (k)	$\boldsymbol{No.of\ problems}$	$\boldsymbol{Problem\ Size}$	$\boldsymbol{Work\ /Cost\ Per\ Problem\ (Print\ and\ Add)}$	$\boldsymbol{Work\ done = Work\ per\ Problem \times No.of\ Problems}$
$\mathbf{0}$	$\mathbf{1}$	$\boldsymbol{n-0=n}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
$\mathbf{1}$	$\mathbf{1}$	$\boldsymbol{n-1}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
$\mathbf{2}$	$\mathbf{1}$	$\boldsymbol{n-2}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
. .	. .	. .	. .	. .
$\boldsymbol{n-1}$	$\mathbf{1}$	$\boldsymbol{n-(n-1)=1}$	$\boldsymbol{C}$	$\boldsymbol{1\times C=C}$
$\boldsymbol{n}$ $(\boldsymbol{Base\ Case})$	$\mathbf{1}$	$\boldsymbol{n-n=0}$ $(\boldsymbol{Base\ Case})$	$\boldsymbol{T(0)}$	$\boldsymbol{1\times T(0)=T(0)}$

$\boldsymbol{Why\ level\ `n`\ is\ Base\ Case?}$

$\boldsymbol{Ans: for, level\ `k`:, Base\ Case = T(0)\ \therefore n-k=0}$

$\boldsymbol{\Rightarrow k=n.}$

$\boldsymbol{Total\ Cost = C+C+C+\cdots(N)times+T(0).}$

$\boldsymbol{=C(N)+T(0)}$

$$We\ know\ , T(0) is\ 1, we\ can\ go\ with\ 1\ , but\ let\ instead\ lets\ write\ T(0) = C_0\ , representing\ constant.$$

$$Total\ Cost = C(N) + C_0$$

$$As, C\ and\ C_0\ both\ are\ constant\ Applying\ Big\ O\ , we\ get:$$

$$= O(N)$$

Best Structure [For Balanced Tree]

$$2T\left(\frac{N-1}{2}\right)+C\,,\ Base\ Case: T(0)=1.$$

Work Per Problem or Cost Per Problem :

```
void destroyNodeRecursively(Node *node)
{
    if (node == nullptr)
        return;

    // 1. Destroy Child List first
    destroyNodeRecursively(node->child);

    // 2. Destroy Next List
    destroyNodeRecursively(node->next);

    // 3. Free current
    cout << "Freeing node: " << node->data << endl;
    free(node);
}
```

When the computer is holding a single node in its hand, it has to spend actual CPU cycles to do the following:

- ***Evaluate the if statement.***
- ***Push characters to the console screen (cout).***
- ***Talk to the operating system to release memory (free).***

And this work is being executed during runtime of recursive calls.

That physical CPU effort is the Work per Problem or Cost Per Problem.

And Cost Per Problem is : C or 1 i.e. Constant.

$\boldsymbol{Level\ (k)}$	$\boldsymbol{No.of\ problems}$	$\boldsymbol{Problem\ Size}$	$\boldsymbol{Work\ /Cost\ Per\ Problem\ (Print\ and\ free\ memory)}$	$\boldsymbol{Work\ done = Work\ per\ Problem \times No.of\ Problems}$
$\mathbf{0}$	$\mathbf{1}$	$\boldsymbol{n}$	$\boldsymbol{C}$	$\mathbf{1} \times \boldsymbol{C} = \boldsymbol{C}$
$\mathbf{1}$	2	$\frac{\boldsymbol{n-1}}{\mathbf{2}}$	$\boldsymbol{C}$	$\mathbf{2} \times \boldsymbol{C} = \mathbf{2}\boldsymbol{C}$
$\mathbf{2}$	4	$\frac{\boldsymbol{n-3}}{\mathbf{4}}$	$\boldsymbol{C}$	$\mathbf{4} \times \boldsymbol{C} = \mathbf{4}\boldsymbol{C}$
$\mathbf{3}$	$\mathbf{8}$	$\frac{\boldsymbol{n-7}}{\mathbf{8}}$	$\boldsymbol{C}$	$\mathbf{8} \times \boldsymbol{C} = \mathbf{8}\boldsymbol{C}$
. .	. .	. .	. .	. .
$\boldsymbol{k}$	$\mathbf{2}^{\boldsymbol{k}}$	$\frac{\boldsymbol{N-(2^k-1)}}{\mathbf{2}^{\boldsymbol{k}}}$	$\boldsymbol{C}$	$\mathbf{2}^{\boldsymbol{k}} \times \boldsymbol{C} = \mathbf{2}^{\boldsymbol{k}}\boldsymbol{C}$
$(\boldsymbol{Base\ Case})$	$\boldsymbol{N+1}$	$(\boldsymbol{Base\ Case})$	$\boldsymbol{T(0)}$	$(\boldsymbol{N+1}) \times \boldsymbol{T(0)}$

$\boldsymbol{Q) Why\ `N+1`\ Base\ Case?}$

$\boldsymbol{The\ "Pointer\ Math"\ Proof}$

Imagine of building a perfectly balanced multi − level linked list

with exactly N real nodes.

1. ***Total Pointers***: ***Every single node in your structure contains exactly two pointers*** (*child* ***and*** *next*). ***Therefore, in a list of N nodes, there are exactly 2N total pointers physically sitting in memory***.

2. ***Connecting the Nodes***: ***To connect N nodes together into a single tree, we have to use exactly*** $N - 1$ ***pointers***.
   (***Think about it***: ***to connect 2 nodes, it need 1 pointer. To connect 3 nodes, need 2 pointers***).

3. ***The Leftovers***: ***If we have 2N total pointers, and we used*** $N - 1$ ***of them to wire the structure together, how many pointers are left pointing to absolutely nothing***?

Let's do the math:

$$2N - (N - 1)$$
$$= 2N - N + 1$$
$$= N + 1$$

What does this mean for your code?

Those leftover pointers are all pointing to *nullptr*.

When your recursive function runs, it visits every single pointer in the entire structure.

- ***It visits the N nodes to do real work*** (***calling*** *free*()).
- ***It blindly follows the leftover pointers and hits*** *nullptr*.

Because there are exactly N + 1 leftover nullptr pointers at the bottom of the structure, the base case (if (node == nullptr)return;) is forced to execute exactly N + 1 times.

The N + 1 count includes ALL the nullptrs combined across BOTH recursive cases. It is the total, absolute sum of every single unused pointer in the entire data structure, whether it was meant to be a next pointer or a child pointer.

Where are the nullptrs hiding?

Every single node create in this balanced structure has exactly 2 pointers.

When our recursive function is at a specific node, it blindly tries to follow both of them:

Traverse(node–> child)

Traverse(node–> next)

Depending on where the node is in the tree, those pointers might point to real data, or they might point to nothing:

1. *The* Leaf *Nodes (The very bottom): If a node is at the very bottom of the structure, it has no children and no next nodes.*

   Both of its pointers are nullptr.

   - *When the function tries to go to child, it hits a nullptr and triggers the base case.*
   - *When the function tries to go to next, it hits a nullptr and triggers the base case again.*
   - *Total contribution to the base case: 2.*

2. ***The*** Half-Full ***Nodes*** (***Near the bottom***): ***If the tree isn't perfectly balanced, you might have a node that has a child, but no next node.***

    - ***The child pointer goes to a real node (continuing the recursion).***
    - ***The next pointer hits a nullptr and triggers the base case.***
    - ***Total contribution to the base case: 1.***

3. ***The*** Full***Nodes*** (***The middle***): ***If a node has both a child and a next, the function follows both to real nodes.***

    - ***Total contribution to the base case: 0.***

Combining these 1, 2 cases = N + 1. i. e. How many times bases cases are hit?

N + 1 times,

Just to summarize that golden rule of binary trees:

- ***We have N real nodes.***
- ***They contain 2N total pointers.***
- ***We use N − 1 of those pointers to connect the tree together.***
- ***We are left with exactly N + 1 nullptr pointers.***

Because our recursive function blindly follows every single pointer until it hits a dead end, the base case if (node == nullptr) is mathematically guaranteed to execute exactly N + 1 times, regardless of the tree's exact shape!

Now we have to derive level `k` first:

$$\textbf{Problem size} = \frac{N - \left(2^k - 1\right)}{2^k}$$

$$\textbf{And we know, } T(0) \textbf{ is our base case .}$$

The tree stops growing $(\textbf{it hits the base case})$ ***when the problem size reaches*** $\mathbf{0}$.

So, to find the final level k, ***we set our pattern equal to*** $\mathbf{0}$ ***and solve for*** k:

$$\frac{N - \left(2^k - 1\right)}{2^k} = 0$$

Multiplying 2^k ***on both the sides***:

$$\Longrightarrow N - \left(2^k - 1\right) = 0$$

Moving the $2^k - 1$ ***to the other side, we get***:

$$\Longrightarrow N = 2^k - 1$$

Add 1 ***to both sides to isolate the exponential term***:

$$\Longrightarrow N + 1 = 2^k$$

Take the base -2 ***logarithm of both sides***.

Because the base of our exponent is $2 (\textbf{from } 2^k)$, ***we use the base*** -2 ***logarithm*** $(\log_2)$ ***to target it***.

$$\Longrightarrow \log_2(N+1) = \log_2 2^k$$

$$\Longrightarrow \log_2(N+1) = \mathrm{k} \times \log_2 2$$

$$\Longrightarrow \log_2(N+1) = k \ [\log_a a = 1]$$

$$\therefore k = \log_2(N+1)$$

Hence full table is:

Level (k)	***No. of problems***	***Problem Size***	***Work /Cost Per Problem (Print and free memory)***	***Work done = Work per Problem × No. of Problems***
0	**1**	n	C	$1 \times C = C$
1	2	$\frac{n-1}{2}$	C	$2 \times C = 2C$
2	4	$\frac{n-3}{4}$	C	$4 \times C = 4C$
3	**8**	$\frac{n-7}{8}$	C	$8 \times C = 8C$
. .	. .	. .	. .	. .
k	2^k	$\frac{N-(2^k-1)}{2^k}$	C	$2^k \times C = 2^k C$
. . .	. . .	. . .	. . .	. . .
$\log_2(N+1)$ (***Base Case***)	$N+1$	$\frac{N-(2^{\log_2(N+1)}-1)}{2^{\log_2(N+1)}} = 0$ (***Base Case***)	$T(0)$	$(N+1) \times T(0)$

$$\frac{N-(2^{\log_2(N+1)}-1)}{2^{\log_2(N+1)}}\ (Base\ Case)$$

$$=\frac{N-(N+1-1)}{N+1}$$

$$=\frac{N-N-1+1}{N+1}$$

$$=\frac{0}{N+1}=0$$

$Also\ , no.of\ problems: 2^k, when\ k=\log_2(N+1) \Longrightarrow 2^{\log_2(N+1)} \Longrightarrow N+1$

$We\ can\ also\ take$ from base level : $\log_2(N+1), Number\ of\ Problem\ is: N+1$.

$$As, 2^k = N+1.$$

$$\left[\begin{matrix} From\ Mathematical\ Terms\ apart\ from \\ Pointer\ terms\ explanation. \end{matrix}\right]$$

$Now\ lets\ calculate$:

$Now\ , we\ can\ see\ growth: C+2C+4C+\cdots+2^{k-1}C+(N+1)\times T(0).$

$Note\ , 2^k = N+1 (Already\ proved\ above)$

$\therefore The\ series\ is\ actually,$

$\Rightarrow C+2C+4C+\cdots+2^{k-1}C+2^k\times T(0)\ [Where\ 2^k=N+1]$

$$\left[\begin{matrix} We\ know\ T(0)\ is\ Constant \\ But\ already\ we\ know\ its\ value. \end{matrix}\right]$$

$Hence\ , C + 2C + 4C + \cdots + 2^{k-1}C + (N+1) \times T(0).$

$\Longrightarrow C\left(1 + 2 + 4 + \cdots + 2^{k-1}\right) + (N+1) \times T(0).$

Applying geometric progression(series) :

$$S(n) = \ ar^0 + ar^1 + ar^2 + \cdots + ar^n = a\left(\frac{r^{n+1} - 1}{r - 1}\right) , where$$

$S(n)$ **is sum of first n terms ,** a **is coefficient and** r **is common ratio between the consecutive terms.**

We get:

$\Longrightarrow C(1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2 + \cdots + 1 \times 2^{k-1}) + (N+1) \times T(0).$

$$\Rightarrow C\left(1 \times \left(\frac{2^{k-1+1} - 1}{2 - 1}\right)\right) + (N+1) \times T(0).$$

or we can directly apply geometric series , $for\ [a \times r^{n-1}]$

$$= \ a\left(\frac{r^n - 1}{r - 1}\right)$$

$$\Rightarrow C\left(1 \times \left(\frac{2^k - 1}{2 - 1}\right)\right) + (N+1) \times T(0).$$

$\Rightarrow C\left(2^k - 1\right) + (N+1) \times T(0).$

$$We\ know\ , k = \log_2(N+1)\ , applying\ in\ the\ Equaltion\ we\ get:$$

$$\Rightarrow C\left(2^{\log_2(N+1)} - 1\right) + (N+1) \times T(0).$$

$$\Rightarrow C((N+1) - 1) + (N+1) \times T(0). \left[Note, a^{log_a b} = b\right]$$

$$\Rightarrow CN + (N+1) \times T(0)$$

$$We\ know\ T(0) = 1\ , we\ can\ apply\ that\ here\ but\ instead\ we\ can\ write,$$

$$T(0) or\ t_0\ as\ C_0\ representing\ constant:$$

$$\Rightarrow CN + (N+1) \times C_0$$

$$Applying\ Big\ O\ , in\ the\ equation\ removing\ all\ the\ constants:$$

$$\Longrightarrow O(CN + (N+1) \times C_0)$$

$$\Longrightarrow O(N) + O(N+1)$$

$$\Longrightarrow O(N) + O(N+1)$$

$$\Longrightarrow O(N) + O(N)$$

$$\Longrightarrow O(N+N)$$

$$\Longrightarrow O(2N)$$

$$\Longrightarrow O(N)$$

Alternatively,

$\boldsymbol{Note:, if\ we\ take\ C\ and\ C_0\ as\ 1\ unit\ time\ , which\ will\ lead\ to\ the\ equation:}$

$= \boldsymbol{N + N + 1}$

$= \boldsymbol{2N + 1}$

$= \boldsymbol{O(N)[Alternatively]}$

****